

# Exercise 10 — Using AI to Help with Testing

---

## Exercise Description

---

Use AI to improve testing skills against the task prioritisation algorithm. Cover: Part 1 understanding what to test, Part 2 improving a single test, Part 3 TDD (a new feature and a bug fix), and Part 4 integration testing.

**Files:** - `models.py` — Task model, extended with an `assignee` field (for the new feature). - `task_priority.py` — scoring, extended via TDD (assignee boost + freshness fix). - `test_task_priority.py` — the full test suite (12 tests, all passing).

---

## Part 1 — Understanding What to Test (behaviour analysis)

---

The behaviours of `calculate_task_score` worth testing, each in isolation:

- **Base priority** — LOW/MEDIUM/HIGH/URGENT → 10/20/40/60.
- **Due-date buckets** — overdue +35, today +20,  $\leq 2$  days +15,  $\leq 7$  days +10, else 0.
- **Status penalties** — DONE -50, REVIEW -15.
- **Tag boost** — +8 once if any of blocker/critical/urgent (even if several match).
- **Freshness** — +5 if updated < 1 day ago.

**Edge cases identified:** the `timedelta.days` truncation at bucket boundaries (a task built "1 day out" can read as 0 days by the time it's scored); a task with two matching tags should still only get +8; DONE tasks can go negative.

`TestScoringComponents` covers each factor; test data pins `updated_days_ago=5` so the freshness boost doesn't leak into unrelated assertions.

## Part 2 — Improving a Single Test

---

A naive test asserts an exact magic number (`assertEqual(score, 55)`), which is brittle and doesn't say *why*. The improved test asserts **behaviour**: a task due within two days scores exactly 15 more than the same task with no due date

(`test_task_due_within_two_days_scores_higher_than_same_task_with_no_due_date`). It expresses the rule, not an opaque total.

## Part 3.1 — TDD: new feature (+12 for the current user)

---

Red → Green → Refactor: 1. **Red:** wrote `test_assigned_to_current_user_gets_plus_12` (and the "someone else" / "no current user" cases) first — they failed (no feature yet). 2.

**Green:** added an optional `assignee` to `Task` and a `current_user` parameter to

`calculate_task_score`, adding +12 when `task.assignee == current_user`. 3. **Refactor:** threaded `current_user` through `sort_tasks_by_importance` / `get_top_priority_tasks` so the boost affects ordering, then added `test_current_user_boost_changes_ordering`.

## Part 3.2 — TDD: bug fix (freshness)

---

The course's bug is that "days since update" is computed wrongly (dividing by ms in JS / wrong `ChronoUnit` in Java). The faithful **Python analogue** is using `timedelta.seconds` (the sub-day remainder, always 0-86399) instead of `.days`. With `.seconds`, a task last updated 3 days ago still looks "fresh" and wrongly earns +5.

TDD process: wrote `test_stale_task_does_not_get_freshness_boost` (a 3-days-old task must score 20, not 25) → this fails against a `.seconds` implementation → fixed the code to use `.days` → test passes. The regression test stays to prevent reintroduction.

## Part 4 — Integration Test

---

`test_sort_and_top_n_order_by_score` builds four tasks with known scores (urgent+overdue = 95, high+soon = 55, low = 10, done+urgent = 10) and asserts `sort_tasks_by_importance` and `get_top_priority_tasks` order them correctly and honour `limit`. `test_current_user_boost_changes_ordering` verifies the assignee feature actually changes ranking when viewed as that user.

## Test run

---

```
Ran 12 tests in 0.004s
OK
```

**Note on flakiness:** my first run failed two due-date tests because I expressed offsets in whole days — the microseconds between building and scoring a task pushed it across a bucket edge (the exact `.days` truncation issue from Exercise 3). I fixed the tests to use **hour-based, mid-bucket offsets**, which is the correct, deterministic way to test boundary logic.

---

## Submission for Exercise 10 — Using AI to help with testing

---

**1. Test plan (Part 1):** unit tests for every scoring factor (base priority, due-date buckets, status penalties, tag boost, freshness), prioritising the boundary/edge cases (truncation, multi-tag, negative scores); unit first, then integration — see `TestScoringComponents`.

**2. Improved unit tests (Part 2):** replaced magic-number assertions with a behaviour assertion (due-soon scores exactly +15 over no-due), documented via a descriptive test name.

**3. TDD implementation and tests (Part 3):** added the `assignee` field + `current_user` +12 boost (feature) and fixed the freshness calculation to use `.days` not `.seconds` (bug), both driven red→green with regression tests. See [task\\_priority.py](#) and [test\\_task\\_priority.py](#).

**4. Integration test (Part 4):** score → sort → top-N verified together, including that the current-user boost changes ordering.

**5. Reflection:** the biggest testing lesson was making boundary tests deterministic — expressing due dates in mid-bucket hours instead of exact days eliminated a real flakiness caused by `timedelta.days` truncation. Full suite: **12 tests, all passing.**